

МИНОБРНАУКИ РОССИИ
Федеральное государственное бюджетное образовательное учреждение
высшего образования

**«САРАТОВСКИЙ НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ
ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
ИМЕНИ Н. Г. ЧЕРНЫШЕВСКОГО»**

Кафедра математической кибернетики и компьютерных наук

**ИСПОЛЬЗОВАНИЕ ГЕНЕТИЧЕСКИХ АЛГОРИТМОВ И
НЕЙРОННЫХ СЕТЕЙ В АНАЛИЗЕ ДЕФОРМАЦИЙ СТОПЫ**

КУРСОВАЯ РАБОТА

студента 2 курса 211 группы
направления 02.03.02 — Фундаментальная информатика и информационные
технологии
факультета КНиИТ
Кочергина Антона Алексеевича

Научный руководитель
ассистент, к. ф.-м. н.

Д. С. Пантелеев

Заведующий кафедрой
доцент, к. ф.-м. н.

С. В. Миронов

Саратов 2026

СОДЕРЖАНИЕ

ВВЕДЕНИЕ	3
1 Методы автоматизации разметки данных для моделирования	4
1.1 Алгоритмы расчета характеристик плантограммы вручную	4
1.1.1 Коэффициент распластанности переднего отдела стопы k_2 ..	4
1.1.2 Индекс Штритера	4
1.2 Алгоритм анализа плантограммы с использованием генетическо- го алгоритма	4
1.2.1 Подготовка данных	4
1.2.2 Генетический алгоритм для подбора параметров алгорит- ма оконтуривания плантограммы	4
1.2.3 Алгоритм расчета контрольных линий и точек плантограммы	4
1.3 Алгоритм анализа плантограммы с использованием нейронных сетей	4
1.3.1 Архитектура сети	4
1.3.2 Обучение сети	4
1.4 Обработка полученных результатов с помощью алгоритмов ком- пьютерного зрения	4
ЗАКЛЮЧЕНИЕ	4
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	5
Приложение А Приложение	5
Приложение Б Коды	5

ВВЕДЕНИЕ

Диагностика деформаций стопы является актуальной проблемой, такие заболевания характеризуются высокой частотой встречаемости среди людей всех возрастных групп...

1 Методы автоматизации разметки данных для моделирования

1.1 Алгоритмы расчета характеристик плантограммы вручную

1.1.1 Коэффициент распластанности переднего отдела стопы k_2

1.1.2 Индекс Штритера

1.2 Алгоритм анализа плантограммы с использованием генетического алгоритма

1.2.1 Подготовка данных

1.2.2 Генетический алгоритм для подбора параметров алгоритма оконтуривания плантограммы

1.2.3 Алгоритм расчета контрольных линий и точек плантограммы

1.3 Алгоритм анализа плантограммы с использованием нейронных сетей

1.3.1 Архитектура сети

1.3.2 Обучение сети

1.4 Обработка полученных результатов с помощью алгоритмов компьютерного зрения

ЗАКЛЮЧЕНИЕ

Актуальность разработки и внедрения цифровых медицинских технологий в настоящее время признается в рамках междисциплинарного консенсуса специалистов.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

ПРИЛОЖЕНИЕ А

Приложение

ПРИЛОЖЕНИЕ Б

Коды

Функция `resize_with_padding` приводит изображение к фиксированному размеру 512×512 пикселей, сохраняя пропорции и добавляя чёрные полосы. Коэффициент масштабирования $scale = \min(t_w/w, t_h/h)$ выбирается по меньшей стороне, чтобы исключить обрезание. Новые размеры `new_w`, `new_h` вычисляются умножением исходных на `scale` с округлением до целого, после чего вызывается `cv2.resize` с интерполяцией `INTER_AREA`, оптимальной для уменьшения. Чёрный холст нужного размера заполняется масштабированным изображением по центру. Смещения `dx` и `dy` рассчитываются как половина разности между целевыми и новыми размерами. Функция возвращает изображение со строго заданными габаритами и неискажёнными пропорциями исходного объекта.

```
1 def resize_with_padding(image, target_size=(512, 512)):
2     """
3     Изменяет размер, сохраняя пропорции, и добавляет черные
4     ↪     ПОЛОСЫ.
5     """
6     h, w = image.shape[:2]
7     tw, th = target_size
8     scale = min(tw / w, th / h)
9     new_w, new_h = int(w * scale), int(h * scale)
10    resized = cv2.resize(image, (new_w, new_h),
11    ↪    interpolation=cv2.INTER_AREA)
12
13    if len(image.shape) == 2:
14        final_image = np.zeros((th, tw), dtype=np.uint8)
15    else:
16        final_image = np.zeros((th, tw, 3), dtype=np.uint8)
```

```

15
16     dx = (tw - new_w) // 2
17     dy = (th - new_h) // 2
18     final_image[dy:dy + new_h, dx:dx + new_w] = resized
19     return final_image

```

Функция `split_feet_smart` разделяет изображение с двумя стопами по промежутку между ними и применяет нормализующие преобразования. Сначала вызывается `remove` для удаления фона, и из альфа-канала *mask* бинаризацией получается чёрно-белая маска. Функция `cv2.findContours` выделяет внешние контуры и оставляет два крупнейших по площади — это левая и правая стопы. Для каждого контура вычисляется ограничивающий прямоугольник `boundingRect`; прямоугольники сортируются по координате *x*. Линия разреза `split_x` проходит посередине между правым краем левого прямоугольника и левым краем правого. Левая стопа вырезается как `src_image[:, :split_x]` и поворачивается на 180°, правая — как `src_image[:, split_x:]` с поворотом на 180° и горизонтальным отражением `cv2.flip`. В режиме отладки на копии изображения отрисовываются линия разреза, ограничивающие прямоугольники и контуры. Функция возвращает кортеж из двух или трёх изображений.

```

1  def split_feet_smart(src_image, debug=False):
2      """
3      Разделяет изображение с двумя стопами по промежутку между
4      ↪ ними.
5      Левая стопа: поворот на 180 градусов.
6      Правая стопа: поворот на 180 градусов + отражение по
7      ↪ горизонтали.
8      """
9      h, w = src_image.shape[:2]
10
11     no_bg = remove(src_image)
12     mask = no_bg[:, :, 3]
13     _, mask_bin = cv2.threshold(mask, 127, 255, cv2.THRESH_BINARY)
14
15     contours, _ = cv2.findContours(mask_bin, cv2.RETR_EXTERNAL,
16     ↪ cv2.CHAIN_APPROX_SIMPLE)

```

```

14     contours = sorted(contours, key=cv2.contourArea,
    ↪     reverse=True)[:2]
15
16     bboxes = sorted([cv2.boundingRect(c) for c in contours],
    ↪     key=lambda b: b[0])
17     left_edge = bboxes[0][0] + bboxes[0][2] # правый край левой
    ↪     стопы
18     right_edge = bboxes[1][0] # левый край правой стопы
19
20     split_x = (left_edge + right_edge) // 2
21
22     # Левая стопа:
23     left_foot = src_image[:, :split_x]
24     left_foot = cv2.rotate(left_foot, cv2.ROTATE_180)
25
26     # Правая стопа:
27     right_foot = src_image[:, split_x:]
28     right_foot = cv2.rotate(right_foot, cv2.ROTATE_180)
29     right_foot = cv2.flip(right_foot, 1)
30
31     if debug:
32         debug_img = src_image.copy()
33         cv2.line(debug_img, (split_x, 0), (split_x, h), (0, 255,
    ↪         0), 3)
34         for x, y, bw, bh in bboxes:
35             cv2.rectangle(debug_img, (x, y), (x + bw, y + bh), (0,
    ↪             0, 255), 2)
36             cv2.drawContours(debug_img, contours, -1, (255, 0, 0), 2)
37             return left_foot, right_foot, debug_img
38
39     return left_foot, right_foot

```